COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	02/09/2026	**DATE SUBMITTED:**	02/09/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 [Searching Techniques]

1. PROCEDURE OUTPUT

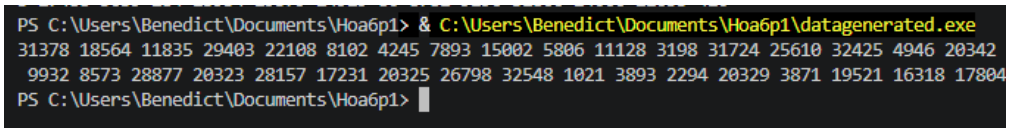
Screenshot	
Observation	The numbers are generated using the rand() function.

Table 6-1. Data Generated and Observations

Code

```
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){

    //while loop
    int i = 0;

    while(i < size) {
        if(dataFind == array[i]) {
            std::cout<<"Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout<<"Searching is unsuccessful\n";


    for (int j = 0; j < size; j++){
        if(dataFind == array[j]) {
            std::cout<<"Searching is successful\n";
            return;
        }
    }
}
```

Output

```
PS C:\Users\Benedict\Documents\Hoa6p1> & C:\Users\Benedict\Documents\Hoa6p1\datagenerated.exe
1905 17498 14384 28199 9290 5244 31748 3392 29053 28325 25730 14874 22074 9173 11005 968 13329
 30189 4400 17170 19096 9205 12738 15774 12655 10815 13235 10656 1840 3557 17220
What number to find:  5244
Searching is successful
PS C:\Users\Benedict\Documents\Hoa6p1>
```

```
PS C:\Users\Benedict\Documents\Hoa6p1> & C:\Users\Benedict\Documents\Hoa6p1\da
2153 15177 28573 22003 6766 8764 6151 10296 5982 19824 14489 18221 693 21510
2586 28364 4727 17022 11712 21671 21984 8347 1195 15720 3925 25677 1947 28608
What number to find:  67
Searching is unsuccessful
PS C:\Users\Benedict\Documents\Hoa6p1>
```

Observations

The code uses a linear search algorithm to find a specified element inside the array. It uses a for loop or a while loop to traverse the array. And if the comparison is evaluated to be true, it prints out a message saying the search is successful otherwise it prints it is not successful.

Table 6-2a. Linear Search for Arrays

Code

```
template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind){
    //repeat the loop until the current node is not NULL

    while (nodes != nullptr){

        //check if the time = current node data
        if(nodes->data == dataFind){
            std::cout<<"The data:  " << nodes->data << " was found successfully.\n";
            return;
        }
        //move on to the next noide
        nodes = nodes ->next;
    }
    //display unsucessful
    std::cout << "Searching is unsuccessful.\n";

}
```

```
int main () {


    //CONNECT THE NODES
    Node<char> *char1 = new_node('J');
    Node<char> *char2 = new_node('U');
    Node<char> *char3 = new_node('L');
    Node<char> *char4 = new_node('I');
    Node<char> *char5 = new_node('A');
    Node<char> *char6 = new_node('N');

    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;

    linearSearchLL(char1, 'L');
}
```

Output

```
PS C:\Users\Benedict\Documents\Hoa6p1> & C:\Users\Benedict\Documents\Hoa6p1\datagenerated.exe
The data:  L was found successfully.
```

Observation

Linear Search using Linked Lists loops through each element and outputs successfully if the char in the linked list that the user is trying to find gets found.

Table 6-2b. Linear Search for Linked List

Code

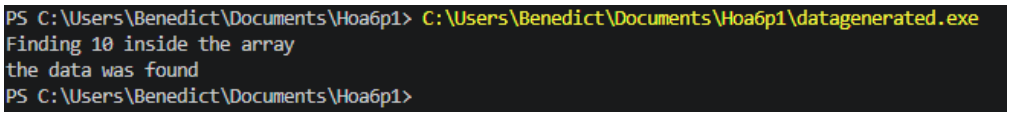
```
//Binary Search
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle;
    while(low <= high){
        middle = (int)((low + high/2));
        if(dataFind == data[middle]){
            std::cout<<"the data was found\n";
            return;
        }
        else if(dataFind < data[middle]){
            high = middle -1;
        }
        else{
            low = middle+1;
        }
    }

    std::cout<<"The data was not found\n";
}
```

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h>
#include "searchingAlgo.h"
const int max_size = 50;

int main () {


    int newArr[max_size] = {1,2,3,4,5,6,7,8,9,10};
    for (int i = 0; i < max_size; i++){
        newArr[i] = i;
    }
    std::cout<<"Finding 10 inside the array\n";
    binarySearchArr(newArr, max_size, 10);
}
```

Output	
Observation	It utilizes binary search to find the given element which in this case is a 10. The algorithm tries to cut the search time in half by looking at the left or right side of a given array. It goes for the middle of the array, and then checks if it is greater or less than the number it is trying to find. If it is greater than the middle, it goes to the right half, otherwise it goes to the left half.

Code

```
template<typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* end){
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != end && fast->next!= end){
        slow = slow->next;

        fast = fast->next->next;
    }

    return slow;
}

template <typename T>
void binarySearchLL(Node<T>* start, T dataFind) {
    Node<T>* last = nullptr;

    while (start != last) {
        Node<T>* middle = getMiddle(start, last);

        if (middle->data == dataFind) {
            std::cout << "Data found\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;
        }
        else {
            start = middle->next;
        }
    }

    std::cout << "Data not found\n";
}
```

datagenerated.cpp > main()

```cpp
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h>
#include "searchingAlgo.h"
const int max_size = 50;

int main () {


   Node<int> *num1 = new_node(10);
    Node<int> *num2 = new_node(20);
    Node<int> *num3 = new_node(30);
    Node<int> *num4 = new_node(40);
    Node<int> *num5 = new_node(50);
    Node<int> *num6 = new_node(60);
    Node<int> *num7 = new_node(70);

    num1->next = num2;
    num2->next = num3;
    num3->next = num4;
    num4->next = num5;
    num5->next = num6;
    num6->next = num7;

    binarySearchLL(num1, 60);

}
```

Output

```
PS C:\Users\Benedict\Documents\Hoa6p1> C:\Users\Benedict\Documents\Hoa6p1\datagenerated.exe
Data found
```

```
PS C:\Users\Benedict\Documents\Hoa6p1> C:\Users\Benedict\Documents\Hoa6p1\datagenerated.exe
Data not found
```

Observation

It utilizes using fast and slow moving pointers to find the middle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

```cpp
// Problem 1: sequential search that RETURNS the number of comparisons
template <typename T>
int linearSearchAR_countCompares(int size, T dataFind, T array[]){
    int comparisons = 0;
    for(int i = 0; i < size; i++){
        comparisons++;                // one comparison per element checked
        if(array[i] == dataFind){
            std::cout << "Searching is successful \n";
            return comparisons;
        }
    }
    std::cout << "Searching is unsuccessful \n";
    return comparisons;
}

template <typename T>
int linearSearchLL_countCompares(Node<T> *nodes, T dataFind){
    int comparisons = 0;
    while(nodes != nullptr){
        comparisons++;
        if(nodes->data == dataFind){
            std::cout << "The data: " << nodes->data << " was found successfully (linked list)\n";
            return comparisons;
        }
        nodes = nodes->next;
    }
    std::cout << "Searching is unsuccessful (linked list)\n";
    return comparisons;
}
```

```cpp
#include <iostream>
#include "node.h"
#include "suppsearchingAlgo.h"

int main(){
    std::cout << "================ PROBLEM 1 ================\n";
    std::cout << "List: [15, 18, 2, 19, 18, 0, 8, 14, 19, 14], key = 18\n\n";

    int arr1[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size1 = 10;

    // Array approach
    int arrComparisons = linearSearchAR_countCompares(size1, 18, arr1);
    std::cout << "Array approach: " << arrComparisons << " comparison(s) needed to find 18.\n\n";

    // Linked list approach (same values, in order)
    Node<int>* head1 = new_node(15);
    Node<int>* n2 = new_node(18);
    Node<int>* n3 = new_node(2);
    Node<int>* n4 = new_node(19);
    Node<int>* n5 = new_node(18);
    Node<int>* n6 = new_node(0);
    Node<int>* n7 = new_node(8);
    Node<int>* n8 = new_node(14);
    Node<int>* n9 = new_node(19);
    Node<int>* n10 = new_node(14);
    head1->next = n2; n2->next = n3; n3->next = n4; n4->next = n5;
    n5->next = n6; n6->next = n7; n7->next = n8; n8->next = n9; n9->next = n10;
    n10->next = nullptr;

    int llComparisons = linearSearchLL_countCompares(head1, 18);
    std::cout << "Linked list approach: " << llComparisons << " comparison(s) needed to find 18.\n";
```

```
================ PROBLEM 1 ================
List: [15, 18, 2, 19, 18, 0, 8, 14, 19, 14], key = 18

Searching is successful
Array approach: 2 comparison(s) needed to find 18.

The data: 18 was found successfully (linked list)
Linked list approach: 2 comparison(s) needed to find 18.
```

Both the array and linked list needed a key value. Linear search algorithm checks every element one by one in the index until it finds the target value, starting from index 0 to the last one. It finds 18 on the array, so the program breaks after finding the target value which is 18

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

```
// Problem 2: sequential search that RETURNS the count of repeating instances of k
template <typename T>
int linearSearchAR_countOccurrences(int size, T k, T array[]){
    int occurrences = 0;
    for(int i = 0; i < size; i++){
        if(array[i] == k){
            occurrences++;
        }
    }
    return occurrences;
}

template <typename T>
int linearSearchLL_countOccurrences(Node<T> *nodes, T k){
    int occurrences = 0;
    while(nodes != nullptr){
        if(nodes->data == k){
            occurrences++;
        }
        nodes = nodes->next;
    }
    return occurrences;
}
```

```
std::cout << "\n================ PROBLEM 2 ================\n";
std::cout << "Count of repeating instances of key = 18 in the same list\n\n";

int arrOcc = linearSearchAR_countOccurrences(size1, 18, arr1);
std::cout << "Array approach: 18 appears " << arrOcc << " time(s).\n";

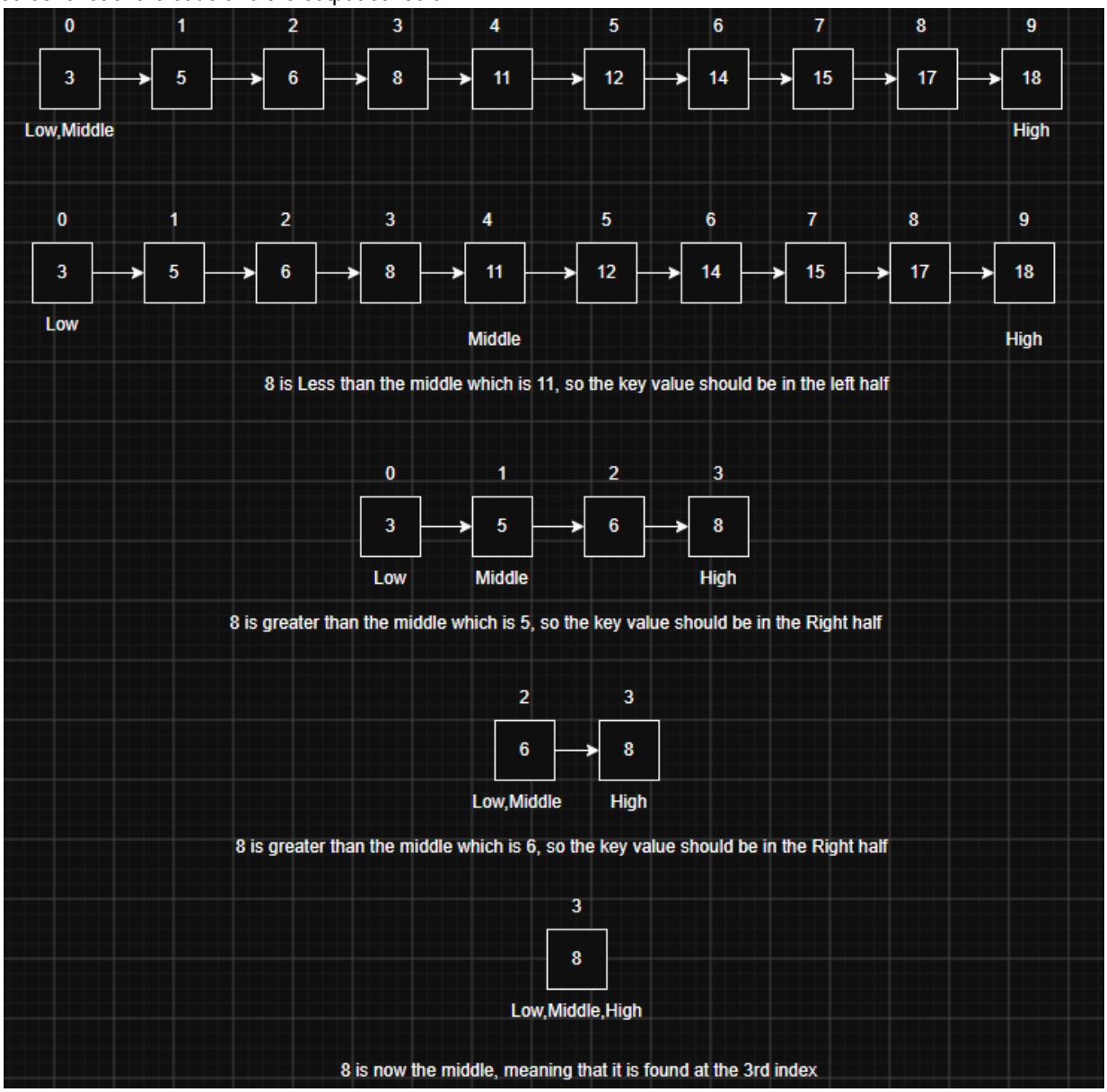
int llOcc = linearSearchLL_countOccurrences(head1, 18);
std::cout << "Linked list approach: 18 appears " << llOcc << " time(s).\n";
```

```
================ PROBLEM 2 ================
Count of repeating instances of key = 18 in the same list

Array approach: 18 appears 2 time(s).
Linked list approach: 18 appears 2 time(s).
```

Both the linked list and array occurrences targeted the key value of 18, which searched the index from start to finish and found the key value on both index 1 and index 4.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console



```cpp
// Problem 3: binary search on an array, tracing every iteration
template <typename T>
void binarySearchArr_traced(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle;
    int iteration = 1;
    while(low <= high){
        middle = (low + high) / 2;
        std::cout << "Iteration " << iteration << ": low=" << low
                  << " high=" << high << " mid=" << middle
                  << " (data[mid]=" << data[middle] << ")\n";
        if(dataFind == data[middle]){
            std::cout << "Search element is found at index " << middle << "!\n";
            return;
        }
        else if(dataFind < data[middle]){
            high = middle - 1;
        }
        else{
            low = middle + 1;
        }
        iteration++;
    }
    std::cout << "Search element is not found\n";
}
```

```cpp
std::cout << "\n================ PROBLEM 3 ================\n";
std::cout << "Sorted list: [3, 5, 6, 8, 11, 12, 14, 15, 17, 18], key = 8\n\n";

int arr2[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int size2 = 10;
binarySearchArr_traced(arr2, size2, 8);
```

```
================ PROBLEM 3 ================
Sorted list: [3, 5, 6, 8, 11, 12, 14, 15, 17, 18], key = 8

Iteration 1: low=0 high=9 mid=4 (data[mid]=11)
Iteration 2: low=0 high=3 mid=1 (data[mid]=5)
Iteration 3: low=2 high=3 mid=2 (data[mid]=6)
Iteration 4: low=3 high=3 mid=3 (data[mid]=8)
Search element is found at index 3!
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

```
// Problem 4: recursive binary search on an array
template <typename T>
void binarySearchArr_recursive(T data[], int low, int high, T dataFind, int iteration = 1){
    if(low > high){
        std::cout << "Search element is not found\n";
        return;
    }
    int middle = (low + high) / 2;
    std::cout << "Iteration " << iteration << ": low=" << low
              << " high=" << high << " mid=" << middle
              << " (data[mid]=" << data[middle] << ")\n";
    if(dataFind == data[middle]){
        std::cout << "Search element is found at index " << middle << "!\n";
        return;
    }
    else if(dataFind < data[middle]){
        binarySearchArr_recursive(data, low, middle - 1, dataFind, iteration + 1);
    }
    else{
        binarySearchArr_recursive(data, middle + 1, high, dataFind, iteration + 1);
    }
}

#endif
```

```
    std::cout << "\n================ PROBLEM 4 (Recursive) ================\n";
    std::cout << "Same sorted list, key = 8, using recursive binary search\n\n";

    binarySearchArr_recursive(arr2, 0, size2 - 1, 8);

    return 0;
}
```

```
================ PROBLEM 4 (Recursive) ================
Same sorted list, key = 8, using recursive binary search

Iteration 1: low=0 high=9 mid=4 (data[mid]=11)
Iteration 2: low=0 high=3 mid=1 (data[mid]=5)
Iteration 3: low=2 high=3 mid=2 (data[mid]=6)
Iteration 4: low=3 high=3 mid=3 (data[mid]=8)
Search element is found at index 3!
```

It returns that the target data 8 was found in the 3rd index, but the main difference between the two function is it instead uses a looping condition to find the target value.

Conclusion
This laboratory activity successfully demonstrated the practical application of linear and binary search algorithms using both array and linked list data structures. Linear search evaluates each element sequentially, making it highly versatile for both data structures. Conversely, binary search efficiently divides the dataset in half repeatedly, making it the ideal choice for sorted arrays in ascending order. Ultimately, learning about both techniques is foundational to understanding data structures and algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.